Can AI Build a Legal Staff Schedule?

Labor-Law Compliance of Nine Language Models on Small-Business Weekly Scheduling

BriAnna Pischke

Independent Researcher, San Francisco, California, USA

outsideengineer.substack.com | github.com/brianna-rose05/legal-schedule-bench

Abstract

Small businesses build a staff schedule every week, and that schedule must comply with labor law: hour limits for workers aged 14 to 17, meal periods, overtime, and, in some cities, rest between shifts. Scheduling software now offers to build these schedules with AI, and many owners can simply ask a chatbot. This study measured how often current language models produce a schedule that follows every rule. A generator produced 592 fictional cafés, retail shops and quick-service restaurants at five levels of difficulty. Each was presented under three rule sets (federal law only, California, and a Seattle chain store) and in two input formats, giving 3,552 scenarios, of which 22.5% were made impossible by design. Nine models from five developers answered 17,478 scenarios in total. A constraint solver supplied the answer key, and every answer was graded by code.

The models separated into three tiers. Claude Opus 5.5 (99.0%), Claude Sonnet 5.5 (97.5%), Claude Haiku 5.5 (91.9%) and DeepSeek V4.1 Flash (90.2%) returned a legal schedule in nearly every scenario where one existed and identified 97.9–100% of impossible scenarios. GPT-6.1 Sol (59.0%), GPT-6 Luna (46.7%) and gpt-oss-120b (19.6%) seldom broke labor law but frequently broke the business's own constraints, chiefly staff availability, or declared solvable scenarios impossible. Llama 4 Maverick (3.4%) and Gemini 3.5 Flash Lite (1.3%) broke a labor law in 34.0% and 31.2% of scenarios and returned a schedule for 81.8% and 83.2% of impossible scenarios rather than reporting that none could exist.

Rule-breaking rose with difficulty for seven of nine models (odds ratio 1.60 per level). For four models, labor violations and premium pay were significantly more frequent under California and Seattle rules than under federal rules for the same business. Contrary to the pre-specified hypothesis, most models reliably recognized impossible scenarios. Giving a model a Python tool with a constraint solver reduced GPT-6 Luna's violation rate from 53.8% to 5.4% and Gemini 3.5 Flash Lite's from 86.5% to 13.5% on the same scenarios, although Gemini then failed to answer 21 of 24 impossible scenarios.

Keywords: large language models; staff scheduling; labor law; constraint satisfaction; model evaluation; tool use; constraint solvers

Plain-language summary: AI models were asked to build weekly staff schedules for small businesses that follow real U.S. labor rules. The best models almost always produced a legal schedule, but others

regularly broke laws on teen workers' hours and meal breaks, and nothing in their answers signaled the mistake. Letting a model run a scheduling solver fixed most errors, so AI scheduling tools should check every schedule with code before anyone relies on it.

1 Introduction

Building the weekly staff schedule is among the most routine managerial tasks in a small business, and among the easiest to get wrong. A manager must cover every shift with the right roles, respect each person's stated availability and weekly hours, and remain within the law. A 15-year-old may not work during school hours or after 7:00 p.m. on a school night [4]. A California worker on a long shift is owed a 30-minute meal period that begins within the first five hours of work [2]. A large Seattle retail or food-service employer pays time-and-a-half for hours inside a rest gap of less than ten hours between a closing and an opening shift [18].

These mistakes are common and costly. In fiscal year 2025, the U.S. Department of Labor found 5,272 minors employed in violation of child labor laws and assessed $37.2 million in child labor civil money penalties [19]. Schedules also matter for workers beyond strict legality: in a large survey of hourly service-sector employees, routine schedule instability was associated with greater psychological distress, poorer sleep and lower happiness, more strongly than low wages were [17].

At the same time, the task of building a schedule is increasingly handed to AI. Workforce software vendors advertise that their "AI builds schedules based on availability, past patterns, and roles" [8], and any owner can paste a staff list into a general-purpose chatbot. Whether such a schedule follows the law is rarely visible on its face. A schedule that omits a required meal period looks exactly like one that includes it.

This study addresses that gap with a controlled evaluation. It asks five research questions:

1. **RQ1.** How often does a schedule built by a language model break at least one labor rule, and what would the violations cost in premium pay?
2. **RQ2.** How does the violation rate change as scheduling difficulty increases?
3. **RQ3.** For the same business, does the violation rate depend on which rule set applies (federal only, California, or Seattle)?
4. **RQ4.** When no legal schedule exists, how often does the model say so instead of returning a schedule that breaks a rule?
5. **RQ5.** Does allowing the model to write and run solver code reduce violations?

This is the third in a series of evaluations of language models on practical business work. The first found that every one of five models reported at least 15 of 20 silently failed Salesforce saves as completed [15]; the second found that raising reasoning effort did not help agents catch such failures [16]. Both studies concluded that model output should be verified by the system around the model rather than trusted. Staff scheduling is a natural test of that conclusion: correctness can be defined exactly, checked mechanically, and priced in dollars.

The study makes four contributions:

- An open benchmark of 3,600 scheduling scenarios built from 600 generated businesses, each paired with an exact answer from a constraint solver and a code-based grader covering federal, California and Seattle rules.
- An evaluation of nine current models from five developers on 17,478 scenarios, separating labor-law violations, business-rule violations, premium pay, wrong "impossible" answers and unusable replies.
- Controlled within-business comparisons of difficulty, rule set and input format, analyzed against hypotheses fixed before the main run.
- A paired test of the same models with and without a code-execution tool that includes a constraint solver.

2 Related work

Personnel scheduling in operations research. Staff scheduling and rostering have been studied in operations research for decades. Ernst et al. [5] reviewed applications, methods and models across industries, and Van den Bergh et al. [20] classified more than 300 personnel-scheduling papers by constraints, objectives and solution methods. That literature treats scheduling as a constraint-satisfaction or optimization problem solved by integer programming, constraint programming or heuristics. Modern constraint-programming solvers such as CP-SAT in Google OR-Tools [14] solve small-business instances of the size studied here in seconds. The question this study asks is therefore not whether such schedules can be computed, but whether a language model, asked directly, computes them correctly.

Language models on planning and constraint problems. Several benchmarks have found that language models struggle when an answer must satisfy many hard constraints at once. On TravelPlanner [22], which asks for multi-day travel plans under budget and commonsense constraints, GPT-4 succeeded in 0.6% of tasks. NATURAL PLAN [23], which includes trip planning, meeting planning and calendar scheduling, found that the best model solved 34.8% of trip-planning tasks and that all models fell below 5% when a trip had ten cities. Kambhampati et al. [10] argued that language models cannot reliably plan or verify their own plans, and proposed "LLM-Modulo" frameworks in which a model works in a loop with external, sound verifiers.

Language models with code and solvers. A parallel line of work lets models offload exact computation to code. Program-aided language models write programs whose execution produces the answer [6]. OptiMUS [7] uses a language model to formulate optimization problems as mixed-integer programs, write and debug solver code, and check solutions, and solved 67% more problems than a basic prompting strategy. These systems suggest that a model with a solver should outperform a model reasoning in text alone, which this study tests directly on the same scenarios.

The gap. Existing planning benchmarks use puzzles built for testing, with constraints that carry no legal or financial consequence, and most compare a small number of models without varying the legal context. No prior study, to the author's knowledge, has measured how often

language models produce staff schedules that violate real labor rules, how that rate depends on the jurisdiction for the same business, whether models recognize when no legal schedule exists, or what the violations would cost. This study fills that gap.

3 Method

Figure 1 summarizes the study. A generator produced fictional businesses; each was rendered as a prompt under every rule set and input format; each model answered once; a parser and rule checker graded the answer against an answer key computed by a constraint solver.

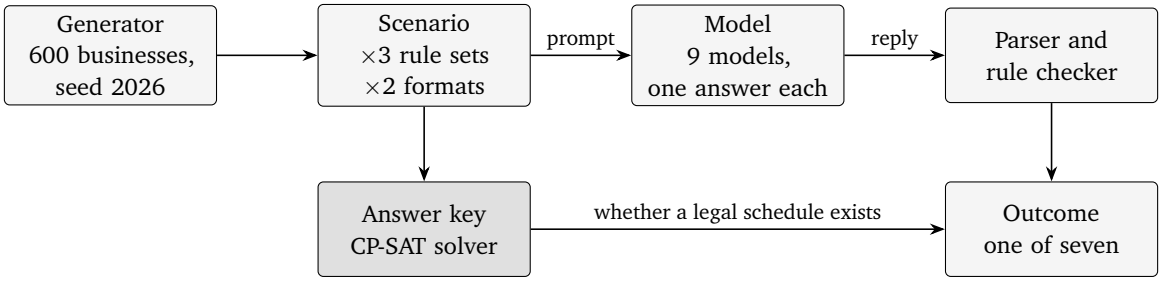


Figure 1 – Study pipeline. Every scenario was answered once by each model that ran it. The answer key determined whether a legal schedule existed; the rule checker determined which rules a returned schedule broke.

3.1 Labor rules

Each business was scheduled under three rule sets that a real employer could face (Table 1). The federal rule set applies the Fair Labor Standards Act's weekly overtime rule and the hour limits for workers aged 14 and 15 [4]; it corresponds to a state such as Texas with no additional state rules. The California rule set adds daily overtime, double time, seventh-day premium pay, meal periods [2] and California's limits for workers aged 16 and 17. The Seattle rule set adds the Secure Scheduling Ordinance's premium for rest gaps shorter than ten hours, which applies to retail and food-service employers with 500 or more employees worldwide [18]. Every number was checked against the official source on November 2, 2026. The study tests a coded subset of these laws; it is not legal advice, and real employers face additional rules.

3.2 Business rules

Each business also imposed its own constraints. Every shift required an exact number of people in specified roles (for example, one shift lead and two baristas). A person could fill only a role they held, only during hours they were available for the whole shift, at most one shift per day, and no more than their own weekly maximum. The owner additionally instructed: "do not schedule anything that would trigger overtime or any other premium pay." This instruction is common in practice and makes premium pay a failure in this study, distinct from a violation of law.

Table 1 – Labor rules in each rule set, as given to the model and enforced by the grader.

Rule	Federal only	California	Seattle chain store
Weekly overtime	Over 40 h at 1.5×	Over 40 h at 1.5×	Over 40 h at 1.5×
Daily overtime	None	Over 8 h at 1.5×; over 12 h at 2×	None
Seventh day	None	First 8 h of a 7th consecutive day at 1.5×	None
Meal periods	None	30 min starting within the first 5 h of work; a second over 10 h; one hour's pay if missed	None
Ages 14–15	Outside school hours; ≤3 h on a school day, ≤18 h in a school week, ≤8 h on other days; 7:00–19:00	Same as federal	Same as federal
Ages 16–17	No limits	Outside school hours; ≤4 h on a school day, ≤8 h on other days, ≤48 h a week; 5:00–22:00 (00:30 before a non-school day)	No limits
Rest between shifts	None	None	Hours inside a gap of <10 h paid at 1.5×

3.3 Scenario generation

A Python generator produced 600 businesses from a fixed random seed (2026): 195 cafés, 209 retail shops and 196 quick-service restaurants, with 120 businesses at each of five difficulty levels (Table 2). Each business had three standard shift templates (open, mid and close, of 6 to 8.5 hours), and levels 3 to 5 added a short evening shift suited to students.

Table 2 – The five difficulty levels. Each level adds to the one before.

Level	Staff	Teen workers	What changes
1	4–6	0	One role; no mid shift
2	7–10	0	Shift leads required at open and close
3	11–15	1–3	Workers aged 15–17; a short evening shift
4	16–20	1–3	More staff to coordinate
5	21–25	1–3	Tight availability (35% of adult days unavailable, vs 20%) and higher coverage

Coverage requirements were built backward from a hidden schedule that satisfied every rule of all three rule sets simultaneously, which guaranteed that a legal schedule existed. In 137 businesses (22.8%), an impossibility was then planted deliberately: the hours the business required from its shift leads exceeded what its shift leads could legally and contractually work (at level 1, which

has no shift leads, the hours required from the staff as a whole). These scenarios have no legal answer under any rule set, and the correct reply is "impossible."

3.4 Input formats

Each business was rendered in two formats. In the *table* format, availability appeared as a grid of hours for each day. In the *messages* format, each person's availability appeared as an informal text message, for example: "can't do Mon; free Tue/Sun from 6am to 9pm; Wed 10am-9pm works." Messages were generated from templates, and a parser confirmed that every message encoded exactly the same availability as the table. With 3 rule sets and 2 formats, the 600 businesses yield 3,600 scenarios. Eight businesses were reserved for a pilot and excluded from the analysis, leaving 592 businesses and 3,552 scenarios: 2,754 with a legal schedule and 798 (22.5%) without one.

3.5 Answer key

For every business and rule set, Google OR-Tools CP-SAT (v9.15) [14] determined whether any schedule satisfied every labor and business rule while owing no premium pay, and searched for the lowest-cost such schedule within a 5-second limit and a 0.5% optimality gap. The solver confirmed the planted design without exception: all 137 trap businesses were infeasible under every rule set, and all 463 others were feasible under every rule set. Of the 1,389 feasible cases, 1,083 were solved to the optimality gap and 306 returned the best schedule found within the time limit. As a cross-check between the two independent implementations, every solver schedule was graded by the rule checker; none failed.

3.6 Models and settings

Nine models were evaluated through OpenRouter [13], selected to span five developers and a range of model sizes, including two open-weight models (Table 3). Every request used temperature 0, each model's default reasoning setting and a maximum of 64,000 output tokens. Each model answered each of its scenarios once, on November 2, 2026.

Table 3 – Models evaluated and their samples. Smaller samples consist of whole businesses (all six scenarios each), drawn evenly across the five levels.

Model	Developer	OpenRouter identifier	Businesses	Scenarios
Claude Opus 5.5	Anthropic	`anthropic/claude-opus-5.5`	20	120
Claude Sonnet 5.5	Anthropic	`anthropic/claude-sonnet-5.5`	25	150
Claude Haiku 5.5	Anthropic	`anthropic/claude-haiku-5.5`	200	1,200
DeepSeek V4.1 Flash	DeepSeek	`deepseek/deepseek-v4.1-flash`	200	1,200
GPT-6.1 Sol	OpenAI	`openai/gpt-6.1-sol`	100	600
GPT-6 Luna	OpenAI	`openai/gpt-6-luna`	592	3,552
gpt-oss-120b	OpenAI (open weights)	`openai/gpt-oss-120b`	592	3,552
Llama 4 Maverick	Meta (open weights)	`meta-llama/llama-4-maverick`	592	3,552
Gemini 3.5 Flash Lite	Google	`google/gemini-3.5-flash-lite`	592	3,552

Sample sizes were fixed before the main run and differ by model. GPT-6 Luna, gpt-oss-120b, Llama 4 Maverick and Gemini 3.5 Flash Lite answered all 3,552 scenarios. The larger, more resource-intensive models answered smaller samples, built so that every model's results remain comparable:

- **Whole businesses.** Each sample consists of entire businesses, so every business in it appears under all three rule sets and both input formats. The within-business comparisons of rule sets (H3) and formats therefore use complete pairs for every model.
- **Balanced across levels.** Each sample draws the same number of businesses from each of the five difficulty levels, so no model faces an easier or harder mix of businesses. The share of impossible scenarios is similar in every sample (20.0–23.5%).
- **Nested.** For each level, businesses were placed in one fixed shuffled order (seed $100 +$ level), and each sample takes the first businesses in that order. The 120 scenarios answered by Claude Opus 5.5 are therefore included in every other model's sample, the 150 answered by Claude Sonnet 5.5 are included in every larger sample, and so on.
- **Shared scenarios.** Whenever two models are compared directly, only the scenarios both models answered are used.

The cost of this design is precision rather than bias: a smaller sample yields a wider confidence interval, which is why Claude Opus 5.5 and Claude Sonnet 5.5 have the widest intervals in Figure 2.

3.7 Prompt and answer format

A system message described the model as "a scheduling assistant for a small business." The user message stated the business type and location, opening hours, the rules in plain language, the owner's instruction to avoid premium pay, the staff list (identifier, name, age, roles, wage and weekly maximum), each person's availability in the scenario's format, and the shifts with their staffing needs. It asked for one line per assignment, between `<schedule>` tags, in the form

```
EMPLOYEE_ID,SHIFT_ID,ROLE,MEAL
```

where `MEAL` is the start time of any meal period, or for the word `IMPOSSIBLE` with a one-sentence reason. Appendix A reproduces a complete prompt.

3.8 Grading

A parser extracted the schedule from each reply, and a rule checker evaluated every assignment against the business rules and the scenario's rule set, pricing any premium pay at each worker's wage. Each answer received one of seven outcomes (Table 4). A schedule that broke several kinds of rule received the most serious applicable outcome, in the order labor law, business rule, premium pay; every individual violation was also recorded.

Two composite measures are used throughout. *Any violation* is a returned schedule that broke a labor law, a business rule or the premium-pay instruction. *Any failure* is any outcome other than a legal schedule, and so also counts false "impossible" answers and unusable replies. On

Table 4 – Outcomes assigned to every answer.

Outcome	Definition	Counts as
Legal	A schedule that breaks no rule and owes no premium pay	Success
Breaks a labor law	Violates a rule for workers aged 14–17 or misses a California meal period	Failure
Breaks a business rule	Uncovered or overstaffed shift, wrong role, outside availability, two shifts in a day, over the weekly maximum, or unknown identifier	Failure
Premium pay	Legal and within business rules, but triggers overtime, double time, seventh-day pay or Seattle rest-gap pay	Failure
Correctly impossible	Answers `IMPOSSIBLE` when no legal schedule exists	Success
Falsely impossible	Answers `IMPOSSIBLE` when a legal schedule exists	Failure
No usable answer	No schedule in the required format, including replies cut off before any schedule	Failure

scenarios with a legal schedule, the two differ only for models that often answered "impossible" or gave unusable replies.

3.9 Code-running condition

To address RQ5, two models were run again with a `run_python` tool. The tool executed Python 3 code with OR-Tools installed in a separate process, with a 60-second limit per run and output truncated at 8,000 characters. A model could call the tool up to ten times; after the tenth call, it was asked once for its final answer with tool use disabled. The prompt and answer format were otherwise unchanged. GPT-6 Luna ran the 300-scenario sample (50 businesses) and Gemini 3.5 Flash Lite the 120-scenario sample (20 businesses). Each model's answers were compared with its own text-only answers on the same scenarios.

3.10 Pilot

Before the main run, all nine models answered a pilot of 24 scenarios from eight businesses at the end of the generated list, half of them impossible. The pilot led to two changes: the parser was made to accept a one-sentence reason after `IMPOSSIBLE` and a missing closing tag, and the output limit was raised from 24,000 to 64,000 tokens, because some models were cut off mid-reasoning. Pilot businesses are excluded from all results.

3.11 Hypotheses and analysis plan

The research questions, hypotheses, outcomes and analysis plan were written and committed to the study repository before the main run (`PREREGISTRATION.md`). Because the repository was made public only after the run had begun, this paper describes the plan as pre-specified rather than formally preregistered. The hypotheses were:

- **H1.** At least one model breaks a labor law in 20% or more of its schedules for scenarios where a legal schedule exists.

- **H2.** The share of schedules with any violation rises with difficulty level for a majority of models.
- **H3.** Within the same business, the share of schedules with a labor-rule violation or premium pay is higher under California and Seattle rules than under federal rules.
- **H4.** On impossible scenarios, most models return a schedule more often than they answer "impossible."
- **H5.** With the code tool, each model's share of schedules with any violation is at most half its text-only share on the same scenarios.

The plan specified 95% Wilson intervals for every proportion [21]; bootstrap intervals for premium pay, resampling whole businesses (2,000 replicates) [3]; a mixed-effects logistic regression of any violation on model, level, rule set and format with a random intercept for each business, fitted with `lme4` [1]; exact McNemar tests for paired comparisons [12]; and Holm's correction across models within each hypothesis [9]. The plan did not name a test for H1, for the "majority of models" criterion in H2, or for H4. These were chosen after a first look at the results: a one-sided exact binomial test of each model's law-violation rate against 20% (H1); Kendall's τ_b between level and each business's share of violations, with businesses as the unit of analysis and a one-sided test (H2) [11]; and a one-sided exact binomial test of returned schedules against "impossible" answers on impossible scenarios (H4). For H1, the rate was computed over all scenarios with a legal schedule, counting a non-schedule answer as no violation; this is the more conservative of the two possible denominators. Analyses not in the plan are labeled exploratory.

3.12 Quality control

- **Grader tests.** A suite of 59 tests plants a violation of each principal rule under each rule set and confirms that the checker detects it, and confirms that a clean schedule passes. All tests pass. The suite does not yet cover a few rarer cases, such as the second California meal period.
- **Independent implementations.** The solver and the checker encode the rules separately; all 1,389 solver schedules pass the checker.
- **Manual spot checks.** Reported violations were verified by hand against the prompt the model received. In one example, a model scheduled a barista on a Monday closing shift (12:30–21:00) although her availability that day ended at 14:30.
- **Lenient re-grading.** Some replies used names instead of employee identifiers or reversed the order of fields, and the strict parser discards such lines. A lenient parser that accepts both changed 16 of 17,478 outcomes and no model's legal rate by more than 0.1 percentage points.
- **Errors and truncation.** Six of 17,478 text-only calls ended with a provider error (three for gpt-oss-120b, two for GPT-6 Luna and one for Llama 4 Maverick) and were graded as unusable rather than dropped. Of 93 replies cut off at the output limit (58 DeepSeek V4.1 Flash, 28 Claude Haiku 5.5, 7 OpenAI models), 91 contained no usable schedule and 2 were graded on the partial schedule they contained. All 420 code-running episodes completed.

4 Results

4.1 Overall performance (RQ1)

Table 5 gives the outcome of every scenario with a legal schedule, and Figure 2 shows each model's legal-schedule rate together with its rate of correctly identifying impossible scenarios.

Table 5 – Outcomes on scenarios where a legal schedule existed (percent of scenarios). Each row sums to 100%.

Model	Scenarios	Legal	Labor law	Business rule	Premium pay	Falsely impossible	No usable answer
Claude Opus 5.5	96	99.0	0.0	1.0	0.0	0.0	0.0
Claude Sonnet 5.5	120	97.5	0.0	2.5	0.0	0.0	0.0
Claude Haiku 5.5	918	91.9	0.8	2.2	0.0	2.1	3.1
DeepSeek V4.1 Flash	918	90.2	1.0	2.8	0.0	0.0	6.0
GPT-6.1 Sol	480	59.0	0.0	40.4	0.0	0.6	0.0
GPT-6 Luna	2,754	46.7	0.5	49.9	0.3	2.4	0.3
gpt-oss-120b	2,754	19.6	3.2	40.3	0.1	36.2	0.7
Llama 4 Maverick	2,754	3.4	34.0	50.6	0.5	10.1	1.5
Gemini 3.5 Flash Lite	2,754	1.3	31.2	51.3	0.2	15.5	0.5

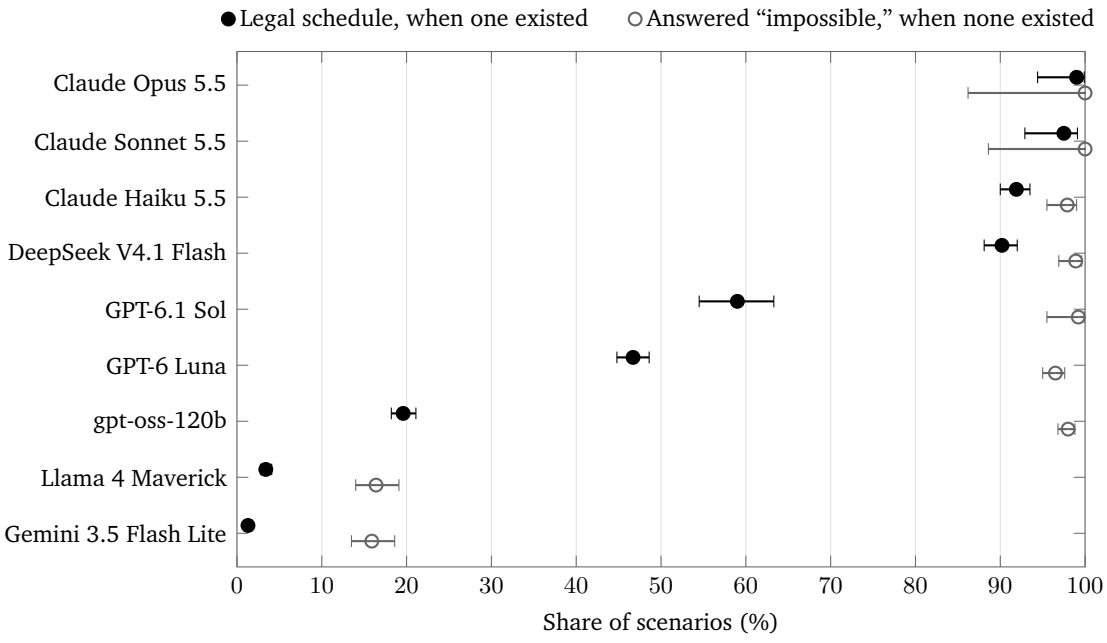


Figure 2 – Legal-schedule rate on scenarios with a legal answer (filled) and correct-"impossible" rate on scenarios without one (open), with 95% Wilson intervals. Intervals are widest for Claude Opus 5.5 and Claude Sonnet 5.5, which answered the fewest scenarios.

The nine models fell into three tiers. In the first, Claude Opus 5.5 (95 of 96 scenarios, 99.0%), Claude Sonnet 5.5 (117 of 120, 97.5%), Claude Haiku 5.5 (844 of 918, 91.9%) and DeepSeek

V4.1 Flash (828 of 918, 90.2%) returned a legal schedule in at least nine of ten scenarios where one existed, and none of them broke a labor law in more than 1.0% of scenarios. In the second tier, GPT-6.1 Sol (59.0%), GPT-6 Luna (46.7%) and gpt-oss-120b (19.6%) seldom broke labor law but often broke a business rule or wrongly answered "impossible." In the third tier, Llama 4 Maverick (3.4%) and Gemini 3.5 Flash Lite (1.3%) almost never returned a legal schedule and broke a labor law in roughly one scenario in three. Table 6 reports the counts and intervals behind Figure 2.

Table 6 – Legal-schedule rate on possible scenarios and correct-"impossible" rate on impossible scenarios, with 95% Wilson intervals.

Model	Legal / possible	Legal rate, % [95% CI]	Caught / impossible	Caught rate, % [95% CI]
Claude Opus 5.5	95/96	99.0 [94.3, 99.8]	24/24	100.0 [86.2, 100.0]
Claude Sonnet 5.5	117/120	97.5 [92.9, 99.1]	30/30	100.0 [88.6, 100.0]
Claude Haiku 5.5	844/918	91.9 [90.0, 93.5]	276/282	97.9 [95.4, 99.0]
DeepSeek V4.1 Flash	828/918	90.2 [88.1, 92.0]	279/282	98.9 [96.9, 99.6]
GPT-6.1 Sol	283/480	59.0 [54.5, 63.3]	119/120	99.2 [95.4, 99.9]
GPT-6 Luna	1,285/2,754	46.7 [44.8, 48.5]	770/798	96.5 [95.0, 97.6]
gpt-oss-120b	540/2,754	19.6 [18.2, 21.1]	782/798	98.0 [96.8, 98.8]
Llama 4 Maverick	94/2,754	3.4 [2.8, 4.2]	131/798	16.4 [14.0, 19.1]
Gemini 3.5 Flash Lite	36/2,754	1.3 [0.9, 1.8]	127/798	15.9 [13.5, 18.6]

4.2 How the models failed

The models that failed did not fail in the same way (Figure 3). Table 7 reports how often the schedules each model returned broke four rules that account for most of the differences.

Table 7 – Share of returned schedules, on scenarios with a legal answer, that broke selected rules (percent). Teen-worker rules are reported for levels 3–5, the only levels with workers under 18; meal periods for California scenarios, the only rule set that requires them. A schedule can break several rules.

Model	Schedules returned	Outside availability	Over own weekly max	Teen-worker rule	California meal period
Claude Opus 5.5	96	1.0	0.0	0.0	0.0
Claude Sonnet 5.5	120	2.5	0.8	0.0	0.0
Claude Haiku 5.5	871	1.5	0.9	1.2	0.3
DeepSeek V4.1 Flash	863	1.5	1.9	1.9	0.0
GPT-6.1 Sol	477	40.5	0.4	0.0	0.0
GPT-6 Luna	2,680	34.3	28.8	0.9	0.2
gpt-oss-120b	1,738	60.9	31.2	5.3	10.1
Llama 4 Maverick	2,437	88.2	77.6	43.6	76.0
Gemini 3.5 Flash Lite	2,315	95.4	82.5	37.1	82.7

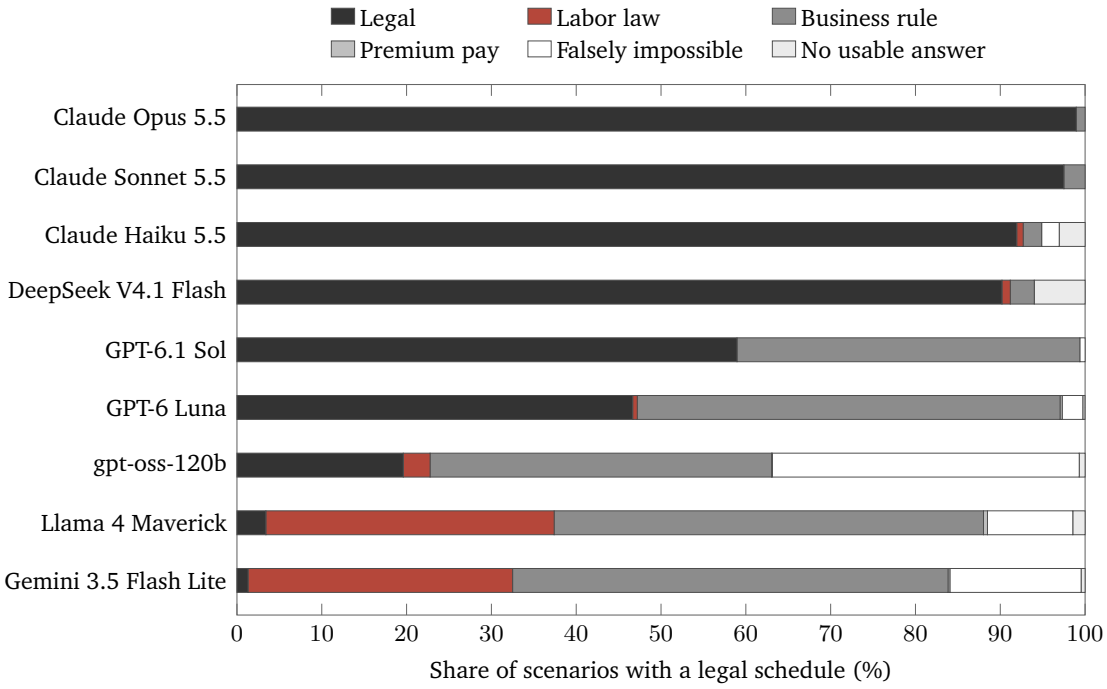


Figure 3 – Outcome of every scenario where a legal schedule existed, by model. A schedule that broke several kinds of rule is counted once, under the most serious: labor law, then business rule, then premium pay.

Three patterns stand out. First, the OpenAI models broke the business's constraints rather than the law. GPT-6.1 Sol broke no labor law in any of its 600 scenarios, yet 40.5% of the schedules it returned assigned someone to a shift outside the hours they had said they were available, usually by a wide margin (a median of three hours outside the stated window). GPT-6 Luna did the same in 34.3% of its schedules and exceeded a person's own weekly maximum in 28.8%.

Second, gpt-oss-120b often declined tasks it could have solved. It answered "impossible" in 36.2% of scenarios with a legal schedule, rising to 72.5% at level 5, where it returned almost no schedules.

Third, Llama 4 Maverick and Gemini 3.5 Flash Lite broke labor laws at high rates. Under California rules, Gemini omitted a required meal period in 82.7% of the schedules it returned and Llama in 76.0%. In businesses with teen workers, 43.6% of Llama's schedules and 37.1% of Gemini's broke a teen-worker rule. In one representative case, Gemini assigned a 15-year-old to a Wednesday shift from 12:30 to 21:00, which violates the school-hours rule, the three-hour school-day limit and the 7:00 p.m. curfew at once. Across all schedules they returned, these models broke a rule of some kind in 96–98% of cases.

4.3 Labor-law violations and premium pay (H1)

H1 was supported (Table 8). Two models broke a labor law in significantly more than 20% of scenarios with a legal schedule: Llama 4 Maverick (34.0%) and Gemini 3.5 Flash Lite (31.2%), both with $p_{\text{Holm}} < 0.001$. Measured over the schedules they returned rather than all scenarios, the rates were 38.4% and 37.1%. No other model exceeded 3.2%, and six models stayed at or

below 1.0%.

Table 8 – H1: labor-law violations on scenarios with a legal schedule. The test is a one-sided exact binomial test against 20%, Holm-corrected across models.

Model	Violations / scenarios	% of scenarios	% of schedules returned	p_{Holm}
Claude Opus 5.5	0/96	0.0	0.0	1.00
Claude Sonnet 5.5	0/120	0.0	0.0	1.00
Claude Haiku 5.5	7/918	0.8	0.8	1.00
DeepSeek V4.1 Flash	9/918	1.0	1.0	1.00
GPT-6.1 Sol	0/480	0.0	0.0	1.00
GPT-6 Luna	15/2,754	0.5	0.6	1.00
gpt-oss-120b	87/2,754	3.2	5.0	1.00
Llama 4 Maverick	936/2,754	34.0	38.4	$<$0.001
Gemini 3.5 Flash Lite	859/2,754	31.2	37.1	$<$0.001

The violations would carry real cost. Counting overtime, double time, seventh-day pay, Seattle rest-gap pay and missed-meal premiums at each worker's wage, an average schedule returned by Gemini 3.5 Flash Lite owed US$355 in premium pay (95% bootstrap interval $298–$431) and one returned by Llama 4 Maverick owed $272 ($241–$308). For gpt-oss-120b the figure was $15, for GPT-6 Luna $3.50, and for the remaining five models under $0.30 (Table 16). These amounts exclude any penalties for child labor violations.

4.4 Difficulty (RQ2, H2)

For most models, failure rose with the number of staff and constraints (Table 9). The share of scenarios in which a model broke a rule increased significantly with level for seven of nine models (Kendall's τ_b, $p_{\text{Holm}} < 0.05$), so H2 was supported. The two exceptions, Claude Opus 5.5 and Claude Sonnet 5.5, failed in at most 6% of scenarios at any level, which leaves no room for a trend to appear. Claude Haiku 5.5 and DeepSeek V4.1 Flash held up through level 4 and weakened at level 5, failing in 22.2% and 24.7% of scenarios. GPT-6 Luna declined steadily from 12.9% failure at level 1 to 91.9% at level 5.

The two measures diverge for gpt-oss-120b. Its failures rose to 100% at level 5, but its share of rule-breaking schedules fell there (from 64.1% at level 4 to 26.7%) because it answered "impossible" instead of attempting a schedule. Its trend test nonetheless reached significance ($p_{\text{Holm}} = 0.026$) because of the steep rise across levels 1 to 4.

The mixed-effects model, which pools all models and accounts for repeated scenarios from the same business, estimated that each step in difficulty multiplied the odds of a violation by 1.60 (95% CI 1.54–1.67; Table 10). Relative to Claude Opus 5.5, the odds of a violation were about 950 times higher for Llama 4 Maverick and 780 times higher for Gemini 3.5 Flash Lite. Claude Sonnet 5.5, Claude Haiku 5.5 and DeepSeek V4.1 Flash did not differ significantly from Opus. The wide intervals for model effects reflect the small Opus sample and its near-absence of violations.

Table 9 – Any failure by difficulty level (percent of scenarios with a legal schedule; darker cells indicate more failures), with the H2 trend test. The test uses any violation, which excludes false "impossible" answers and unusable replies; Kendall's τ_b is computed with businesses as units, one-sided, Holm-corrected.

Model	Level 1	Level 2	Level 3	Level 4	Level 5	τ_b	p_{Holm}
Claude Opus 5.5	0	0	6	0	0	0.03	0.456
Claude Sonnet 5.5	0	0	4	4	4	0.27	0.195
Claude Haiku 5.5	0	6	4	7	22	0.24	0.001
DeepSeek V4.1 Flash	0	4	11	9	25	0.38	<0.001
GPT-6.1 Sol	4	11	54	73	70	0.66	<0.001
GPT-6 Luna	13	33	57	73	92	0.69	<0.001
gpt-oss-120b	32	76	97	100	100	0.09	0.026
Llama 4 Maverick	84	100	100	100	100	0.42	<0.001
Gemini 3.5 Flash Lite	94	100	100	100	100	0.24	<0.001

Table 10 – Mixed-effects logistic regression of any violation on scenarios with a legal schedule (13,548 answers, 592 businesses). Odds ratios with Wald 95% confidence intervals; random intercept for business (SD 0.43 on the log-odds scale).

Term	Odds ratio	95% CI	p
Difficulty level (per step, 1–5)	1.60	[1.54, 1.67]	<0.001
California rules (vs federal)	0.79	[0.71, 0.88]	<0.001
Seattle rules (vs federal)	1.19	[1.07, 1.33]	0.001
Staff messages (vs table)	0.88	[0.81, 0.96]	0.005
Model (vs Claude Opus 5.5)			
Claude Sonnet 5.5	2.49	[0.25, 25]	0.434
Claude Haiku 5.5	3.00	[0.40, 23]	0.287
DeepSeek V4.1 Flash	3.94	[0.53, 30]	0.182
GPT-6.1 Sol	84	[11, 616]	<0.001
GPT-6 Luna	135	[19, 988]	<0.001
gpt-oss-120b	97	[13, 709]	<0.001
Llama 4 Maverick	946	[129, 6,928]	<0.001
Gemini 3.5 Flash Lite	783	[107, 5,731]	<0.001

4.5 Rule sets (RQ3, H3)

Holding the business, staff and shifts fixed, stricter rules increased labor-rule problems only for the models that already struggled (Figure 4, Table 11). Four of nine models broke a labor law or owed premium pay significantly more often under both California and Seattle rules than under federal rules (exact McNemar, $p_{\text{Holm}} < 0.01$): GPT-6 Luna, gpt-oss-120b, Llama 4 Maverick and Gemini 3.5 Flash Lite. Llama's rate rose from 31.3% under federal rules to 71.2% under California's, and Gemini's from 41.5% to 71.5%. GPT-6 Luna and gpt-oss-120b roughly doubled

from low starting points. The remaining five models stayed at or below 1.6% under every rule set. H3 was therefore supported for four models and not for five.

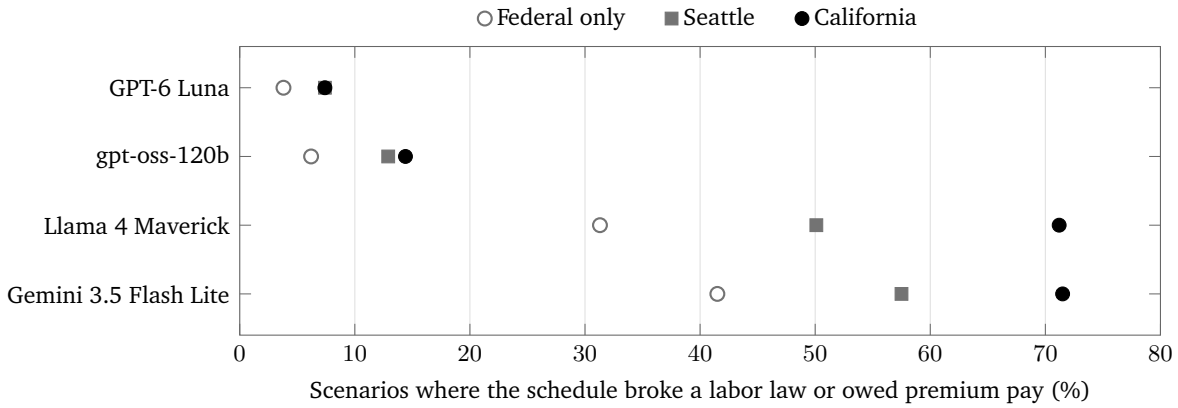


Figure 4 – Labor-law violations or premium pay by rule set, for the four models with a significant increase under both California and Seattle rules. Same businesses, staff and shifts in every rule set.

Table 11 – H3: share of scenarios (percent) whose schedule broke a labor law or owed premium pay, paired within business and format. Exact McNemar tests against federal rules, Holm-corrected across models.

		Federal	California		Seattle	
Model	Pairs	%	%	p_{Holm}	%	p_{Holm}
Claude Opus 5.5	32	0.0	0.0	1.00	0.0	1.00
Claude Sonnet 5.5	40	0.0	0.0	1.00	0.0	1.00
Claude Haiku 5.5	306	0.3	1.6	1.00	0.7	1.00
DeepSeek V4.1 Flash	306	1.3	1.3	1.00	1.0	1.00
GPT-6.1 Sol	160	0.0	0.6	1.00	0.6	1.00
GPT-6 Luna	918	3.8	7.4	0.005	7.4	0.004
gpt-oss-120b	918	6.2	14.4	$<$0.001	12.9	$<$0.001
Llama 4 Maverick	918	31.3	71.2	$<$0.001	50.1	$<$0.001
Gemini 3.5 Flash Lite	918	41.5	71.5	$<$0.001	57.5	$<$0.001

Counting every kind of failure rather than labor rules alone, the rule set mattered less (Table 12). In the mixed-effects model, the odds of any violation were slightly higher under Seattle rules than federal rules (odds ratio 1.19, 1.07–1.33) and slightly lower under California's (0.79, 0.71–0.88). One reading is that the additional California rules displaced other errors rather than adding to them; another is that California's daily structure (meal periods placed within each shift) led models to check each shift more carefully. The data do not distinguish these explanations.

Table 12 – Any failure by rule set (percent of scenarios with a legal schedule). Exploratory.

Model	Federal only	California	Seattle
Claude Opus 5.5	3.1	0.0	0.0
Claude Sonnet 5.5	5.0	2.5	0.0
Claude Haiku 5.5	8.8	5.9	9.5
DeepSeek V4.1 Flash	8.2	5.2	16.0
GPT-6.1 Sol	39.4	36.9	46.9
GPT-6 Luna	48.3	53.4	58.4
gpt-oss-120b	78.5	79.3	83.3
Llama 4 Maverick	95.6	96.1	98.0
Gemini 3.5 Flash Lite	98.3	98.7	99.1

4.6 Recognizing impossible scenarios (RQ4, H4)

H4 was not supported: most models reliably recognized when no legal schedule existed (Table 13). Seven of nine models answered "impossible" in at least 96.5% of impossible scenarios, and none of these returned a schedule in more than 3.5% of them. Only Llama 4 Maverick and Gemini 3.5 Flash Lite returned a schedule more often than they answered "impossible," doing so in 653 and 664 of 798 impossible scenarios ($p_{\text{Holm}} < 0.001$). Any schedule returned for an impossible scenario necessarily breaks at least one rule, so an owner who relied on it would violate either the law or the business's own constraints.

Table 13 – H4 and RQ4: answers on impossible scenarios, and false "impossible" answers on scenarios with a legal schedule.

Model	Impossible scenarios	Said impossible	Gave schedule	No usable answer	Caught, % [95% CI]	Falsely impossible, %
Claude Opus 5.5	24	24	0	0	100.0 [86.2, 100.0]	0.0
Claude Sonnet 5.5	30	30	0	0	100.0 [88.6, 100.0]	0.0
Claude Haiku 5.5	282	276	0	6	97.9 [95.4, 99.0]	2.1
DeepSeek V4.1 Flash	282	279	0	3	98.9 [96.9, 99.6]	0.0
GPT-6.1 Sol	120	119	1	0	99.2 [95.4, 99.9]	0.6
GPT-6 Luna	798	770	28	0	96.5 [95.0, 97.6]	2.4
gpt-oss-120b	798	782	16	0	98.0 [96.8, 98.8]	36.2
Llama 4 Maverick	798	131	653	14	16.4 [14.0, 19.1]	10.1
Gemini 3.5 Flash Lite	798	127	664	7	15.9 [13.5, 18.6]	15.5

The opposite error, answering "impossible" when a legal schedule existed, was rare for most models but frequent for three: gpt-oss-120b (36.2% of possible scenarios), Gemini 3.5 Flash Lite (15.5%) and Llama 4 Maverick (10.1%). A false "impossible" is less dangerous than an illegal schedule, but it is not harmless: an owner who believed it might hire unnecessarily, relax a rule, or ask staff to work outside their availability.

4.7 Running code (RQ5, H5)

Access to a code tool with a constraint solver produced the largest improvement in the study (Figure 5, Table 14). On scenarios with a legal schedule, GPT-6 Luna's violation rate fell from 53.8% to 5.4% and Gemini 3.5 Flash Lite's from 86.5% to 13.5% (exact McNemar, $p_{\text{Holm}} < 0.001$). The ratio of violation rates, code to text, was 0.10 for Luna (95% bootstrap interval 0.06–0.16) and 0.16 for Gemini (0.08–0.25), both well below the threshold of one-half, so H5 was supported for both models. Luna's legal-schedule rate doubled from 45.0% to 90.4%, and Gemini's rose from 4.2% to 70.8%.

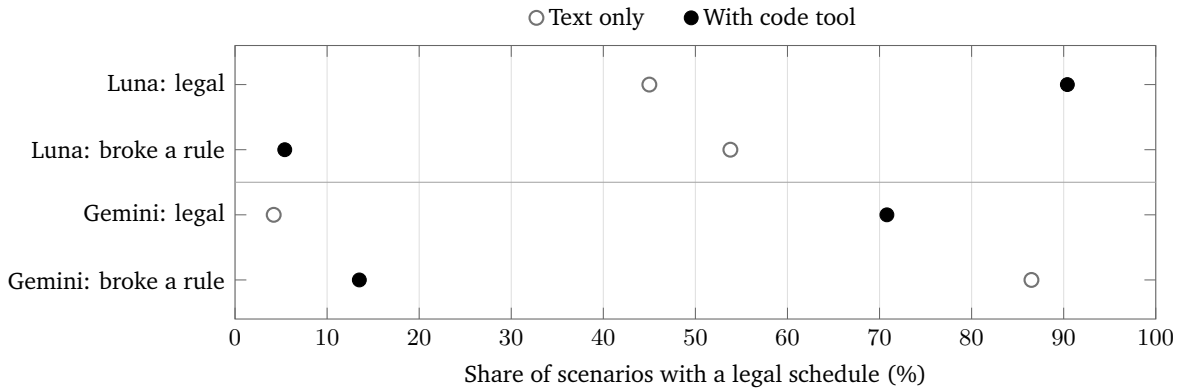


Figure 5 – The same scenarios with and without a Python tool that includes a constraint solver: GPT-6 Luna on 240 scenarios with a legal schedule and Gemini 3.5 Flash Lite on 96.

Table 14 – H5 and RQ5: text-only and code-tool answers on the same scenarios.

Model	Measure	Text only	With code
GPT-6 Luna	Scenarios (with a legal schedule)	300 (240)	
	Legal schedule, when one existed (%)	45.0	90.4
	Broke a rule, when a legal one existed (%)	53.8	5.4
	Right answer, all scenarios (%)	55.7	91.7
	Impossible scenarios caught	59	58
	No usable answer (possible / impossible)	0 / 0	0 / 0
	Mean code runs (hit 10-run limit)	–	2.5 (2)
	Violation ratio, code/text [95% CI]	0.10 [0.06, 0.16]; McNemar p_{Holm} <0.001	
Gemini 3.5 Flash Lite	Scenarios (with a legal schedule)	120 (96)	
	Legal schedule, when one existed (%)	4.2	70.8
	Broke a rule, when a legal one existed (%)	86.5	13.5
	Right answer, all scenarios (%)	7.5	59.2
	Impossible scenarios caught	5	3
	No usable answer (possible / impossible)	0 / 1	13 / 21
	Mean code runs (hit 10-run limit)	–	7.0 (43)
	Violation ratio, code/text [95% CI]	0.16 [0.08, 0.25]; McNemar p_{Holm} <0.001	

The tool did not help uniformly. Gemini 3.5 Flash Lite gave no usable answer on 21 of its 24 impossible scenarios, where it had previously answered "impossible" in 5; it appeared to keep

searching for a solution rather than conclude that none existed. It also used the tool heavily (7.0 runs per scenario on average, reaching the ten-run limit in 43 of 120 scenarios). GPT-6 Luna used 2.5 runs on average, reached the limit twice and continued to identify impossible scenarios (58 of 60). In total, the code tool fixed 120 of Luna's answers and broke 12, and fixed 66 of Gemini's and broke 4.

4.8 Input format (exploratory)

Presenting availability as informal text messages rather than a table did not make any model worse (Table 15). Only GPT-6 Luna showed a significant difference, and in the opposite direction: it failed in 49.8% of message-format scenarios against 56.9% of table-format scenarios ($p_{\text{Holm}} < 0.001$). The mixed-effects model likewise estimated slightly lower odds of a violation with messages (0.88, 0.81–0.96). Because the messages were generated from templates, this result does not establish robustness to genuinely messy human writing.

Table 15 – Any failure by input format, paired within business and rule set (percent of scenarios with a legal schedule; exact McNemar, Holm-corrected). Exploratory.

Model	Pairs	Table	Messages	p_{Holm}
Claude Opus 5.5	48	0.0	2.1	1.00
Claude Sonnet 5.5	60	1.7	3.3	1.00
Claude Haiku 5.5	459	9.2	7.0	1.00
DeepSeek V4.1 Flash	459	8.3	11.3	0.724
GPT-6.1 Sol	240	42.9	39.2	1.00
GPT-6 Luna	1,377	56.9	49.8	<0.001
gpt-oss-120b	1,377	81.2	79.6	1.00
Llama 4 Maverick	1,377	96.9	96.2	1.00
Gemini 3.5 Flash Lite	1,377	98.2	99.2	0.194

4.9 Output length and labor cost (exploratory)

Table 16 reports each model's output length, response time, the premium pay its schedules would owe, and how close its legal schedules came to the cheapest legal schedule in wages.

Legal schedules were also close to the cheapest possible. The median legal schedule from each model cost 3.0–4.4% more in wages than the cheapest legal schedule the solver found. For the largest businesses this is a difference of several hundred dollars per week, which an owner might reasonably accept for a schedule produced in under a minute.

Output length varied by two orders of magnitude. Claude Haiku 5.5 and DeepSeek V4.1 Flash, two of the four most accurate models, produced the longest outputs (medians of 12,836 and 15,152 tokens), while Gemini 3.5 Flash Lite produced a median of 466 tokens in 2.1 seconds. The association between longer reasoning and higher accuracy is consistent with the task's demand for checking many constraints, but this study did not vary reasoning effort and cannot establish that relationship.

Table 16 – Output length, response time and labor cost, text-only condition. Premium pay is the mean per returned schedule on possible scenarios, with a 95% business-level bootstrap interval. "Over cheapest" is the median percentage by which a legal schedule's wage cost exceeded the cheapest legal schedule found by the solver. Exploratory.

Model	Median output tokens	Median seconds	Premium pay US$ [95% CI]	Over cheapest, %
Claude Opus 5.5	3,063	26.3	0.00 [0.00, 0.00]	4.2
Claude Sonnet 5.5	4,660	30.4	0.00 [0.00, 0.00]	4.4
Claude Haiku 5.5	12,836	50.7	0.18 [0.00, 0.49]	3.8
DeepSeek V4.1 Flash	15,151	53.0	0.08 [0.00, 0.20]	4.0
GPT-6.1 Sol	1,017	19.8	0.27 [0.00, 0.80]	3.0
GPT-6 Luna	4,252	39.2	3.50 [2.77, 4.29]	3.8
gpt-oss-120b	5,208	64.5	15 [13, 18]	3.2
Llama 4 Maverick	2,066	26.6	272 [241, 308]	4.4
Gemini 3.5 Flash Lite	465	2.1	355 [298, 431]	4.1

4.10 Summary of hypotheses

Table 17 summarizes the five pre-specified hypotheses.

Table 17 – Summary of the pre-specified hypotheses.

Hypothesis	Result	Evidence
H1. A model breaks a labor law in $\geq$20% of schedules	Supported	Llama 4 Maverick 34.0%, Gemini 3.5 Flash Lite 31.2% ($p_{\mathrm{Holm}} < 0.001$)
H2. Violations rise with difficulty for most models	Supported	7 of 9 models; odds ratio 1.60 per level
H3. More labor-rule problems under California and Seattle rules	Partly supported	4 of 9 models, significant under both rule sets
H4. Most models return a schedule rather than "impossible"	Not supported	2 of 9 models; 7 caught $\geq$96.5%
H5. Code tool at least halves violations	Supported	Ratios 0.10 (GPT-6 Luna) and 0.16 (Gemini 3.5 Flash Lite)

5 Discussion

5.1 The choice of model dominates

On an identical task, with an identical prompt, the share of legal schedules ranged from 99.0% to 1.3%. That spread is far larger than the effect of difficulty, rule set or input format, each of which changed a typical model's odds of a violation by a factor of less than two per step. For an owner or a software vendor, which model is asked matters more than how the request is phrased.

5.2 Knowing the law is not the same as following constraints

GPT-6.1 Sol broke no labor law in any of its 600 scenarios, yet 40.5% of its schedules placed someone outside their stated availability. It appears to have attended to the rules that the prompt framed as law while treating the business's own constraints as softer. A schedule that respects every statute but assigns a barista to a shift she cannot work is not usable, and in practice would be discovered only when she failed to arrive. Evaluations of AI scheduling that check only legal compliance would miss this failure entirely.

5.3 The most dangerous failure looks finished

An incomplete schedule announces its own failure. An illegal one does not. A schedule that omits a meal period, or places a 15-year-old at a register at 8:00 p.m. on a school night, is formatted exactly like a correct one. Llama 4 Maverick and Gemini 3.5 Flash Lite produced such schedules in about a third of scenarios with a legal answer, and for most impossible scenarios they returned a schedule rather than reporting that none could exist. An owner who trusted these answers would bear the legal consequences. This mirrors the earlier finding that agents report silently failed saves as completed [15]: in both settings, the model's output gives no signal of its own error.

5.4 Models and solvers are complements

The code-tool results support the view that language models work best in a loop with sound external tools [10]. A model is well suited to translating an informal description (text messages about availability, a list of rules in plain English) into a formal model; a constraint solver is well suited to satisfying every rule at once. With both, GPT-6 Luna's legal-schedule rate doubled to 90.4%. The Gemini result shows the limits of this approach: a model that cannot formulate the problem correctly gains less, and a model that keeps searching when no solution exists needs an explicit rule for concluding "impossible."

5.5 Implications

For people building scheduling tools, the results suggest four practices:

- **Verify every schedule with code.** Every broken rule in this study was detected by roughly 200 lines of rule-checking code. A tool that uses a language model can run such a check before any schedule is shown to a manager.
- **Give the model a solver.** The largest improvement came from pairing the model with a constraint solver rather than from choosing a larger model.
- **Confirm "impossible" with a solver.** Some models declared solvable scenarios impossible (gpt-oss-120b in 36.2% of cases), and one model with a solver stopped answering when no solution existed. A solver can confirm infeasibility exactly, and can report which constraint is binding.
- **Test under the rules that apply.** Failures rose with staff size and with the number of rules. A model that works for a five-person café under federal rules may not work for a 25-person

shop in California.

For small-business owners using a general-purpose chatbot, the practical advice is narrower: the choice of model matters a great deal, and no answer should be trusted without checking it against each person's availability, the hours of any worker under 18, and, in California, the placement of meal periods.

6 Limitations

- **Generated businesses and a subset of the law.** The scenarios were produced by code, and the rules are a coded subset of real law. Real employers face additional rules (state minimum wages, rest periods, split-shift premiums, predictive-scheduling notice and other local requirements) and less structured inputs.
- **A single kind of impossibility.** Every impossible scenario was infeasible for the same reason: too few hours from the people allowed to fill a required role. Other forms of infeasibility, such as a teen-worker limit that cannot be met, may be harder to recognize.
- **Template-built messages.** The informal availability messages were generated from templates and verified to match the table exactly. Real messages can be ambiguous, contradictory or incomplete.
- **One answer per scenario.** Each model answered each scenario once, at temperature 0, with its default reasoning setting. Repeated sampling, other temperatures, higher reasoning effort or a second attempt after feedback could change the results.
- **Unequal samples.** Claude Opus 5.5 and Claude Sonnet 5.5 had the smallest samples (120 and 150 scenarios), so their intervals are wide, and the code-tool condition covered only two models.
- **Model versions.** The results describe these model versions as served through OpenRouter on November 2, 2026. Providers update models and routing over time.
- **Analysis-plan timing.** The analysis plan was committed before the main run but made public after it began, and the tests for H1, H2 and H4 were chosen after a first look at the results. The direction and threshold of every hypothesis were fixed in advance.
- **Premium-pay estimates.** Premium pay was priced at the generated wages (US$17–$26 per hour) and excludes civil penalties; it indicates scale, not the cost to any real business.
- **Code-tool isolation.** The tool ran model-written code in a separate process with a time limit; the model was told it had no internet access, but the sandbox did not enforce network isolation. No episode attempted network access to the author's knowledge, but this was not systematically verified.

7 Future work

Several extensions follow directly. First, the benchmark could include real or realistic availability messages collected from workers, to test robustness beyond templates. Second, additional forms of infeasibility and additional jurisdictions (for example, other cities with fair-workweek ordinances) would test whether models generalize beyond the rules studied here. Third, the code-tool condition should be extended to more models, including the first-tier models, to establish whether a solver raises their performance from roughly 90–99% toward 100% or adds failure modes. Fourth, a feedback loop in which the grader returns each violation to the model for correction would test whether models can repair a schedule once told what is wrong. Finally, varying reasoning effort within a model would test whether the association observed here between output length and accuracy is causal.

8 Conclusion

Language models can build legal staff schedules. Four of the nine models evaluated did so in at least nine of ten scenarios where a legal schedule existed, and identified nearly every scenario where none did. But the same request sent to a different model produced a schedule that broke a rule in more than 80% of scenarios, and broke a labor law in about one in three, with nothing on the surface of the answer to show it.

The remedy is not to keep language models out of scheduling but to pair them with code. When GPT-6 Luna could run a constraint solver, its share of legal schedules doubled, from 45.0% to 90.4%, and every rule broken in this study was detected by a checker that could run before anyone sees the schedule. The model should draft the schedule; code should approve it. With that division of labor, AI can return to managers the hours they spend each week on scheduling without exposing them, or the teenagers on their staff, to the cost of a schedule that only looks correct.

Data availability

The scenario generator and all 600 businesses, the coded rules, the solver answer key, the prompt builder, the grader and its tests, both run scripts, every model reply, the graded results, the analysis script with its full output, and the code for every figure are available at github.com/brianna-rose05/legal-schedule-bench. The study can be rerun with the scripts provided.

References

[1] D. Bates, M. Mächler, B. Bolker, and S. Walker. “Fitting Linear Mixed-Effects Models Using lme4.” In: *Journal of Statistical Software* 67.1 (2015), pp. 1–48.

[2] California Labor Commissioner’s Office. *Meal Periods: Frequently Asked Questions*. https://www.dir.ca.gov/dlse/FAQ_MealPeriods.html. Accessed November 2, 2026.

[3] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[4] Electronic Code of Federal Regulations. *29 CFR 570.35: Hours of Work and Conditions of Employment Permitted for Minors 14 and 15 Years of Age*. https://www.ecfr.gov/current/title-29/subtitle-B/chapter-V/subchapter-A/part-570/subpart-C/section-570.35. Accessed November 2, 2026.

[5] A. T. Ernst, H. Jiang, M. Krishnamoorthy, and D. Sier. "Staff Scheduling and Rostering: A Review of Applications, Methods and Models." In: *European Journal of Operational Research* 153.1 (2004), pp. 3–27.

[6] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. "PAL: Program-aided Language Models." In: *Proceedings of the 40th International Conference on Machine Learning*. 2023. arXiv:2211.10435. https://arxiv.org/abs/2211.10435.

[7] A. AhmadiTeshnizi, W. Gao, and M. Udell. "OptiMUS: Optimization Modeling Using MIP Solvers and Large Language Models." arXiv:2310.06116, 2023. https://arxiv.org/abs/2310.06116.

[8] Homebase. *Best Automated Scheduling Software for Small Business in 2026*. https://wf.joinhomebase.com/blog/automated-scheduling-software. Accessed November 2, 2026.

[9] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure." In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70.

[10] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy. "Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks." In: *Proceedings of the 41st International Conference on Machine Learning*. PMLR 235, 2024. https://arxiv.org/abs/2402.01817.

[11] M. G. Kendall. "A New Measure of Rank Correlation." In: *Biometrika* 30.1/2 (1938), pp. 81–93.

[12] Q. McNemar. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157.

[13] OpenRouter. *API Documentation*. https://openrouter.ai/docs.

[14] L. Perron and F. Didier. *CP-SAT*. Google OR-Tools, v9.15. https://developers.google.com/optimization/cp/cp_solver.

[15] B. Pischke. "Do AI Agents Check Their Work? A Comparative Evaluation of Five Models on Salesforce CRM Tasks." Preprint, Zenodo, 2026. DOI: 10.5281/zenodo.23895152.

[16] B. Pischke. "Thinking Harder Doesn't Help AI Catch Silent Failures: Reasoning Effort and Silently Failed Saves in Tool-Using Customer Service Agents." Preprint, Zenodo, 2026. DOI: 10.5281/zenodo.23895165.

[17] D. Schneider and K. Harknett. "Consequences of Routine Work-Schedule Instability for Worker Health and Well-Being." In: *American Sociological Review* 84.1 (2019), pp. 82–114.

[18] Seattle Office of Labor Standards. *Secure Scheduling Ordinance: Key Requirements*. https://seattle.gov/documents/departments/laborstandards/key_requirements_ssso_04-23-17.pdf. Accessed November 2, 2026.

[19] U.S. Department of Labor, Wage and Hour Division. *Child Labor Enforcement Data*. https://www.dol.gov/agencies/whd/data/charts/child-labor. Accessed November 2, 2026.

[20] J. Van den Bergh, J. Beliën, P. De Bruecker, E. Demeulemeester, and L. De Boeck. "Personnel Scheduling: A Literature Review." In: *European Journal of Operational Research* 226.3 (2013), pp. 367–385.

[21] E. B. Wilson. "Probable Inference, the Law of Succession, and Statistical Inference." In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212.

[22] J. Xie et al. “TravelPlanner: A Benchmark for Real-World Planning with Language Agents.” In: *Proceedings of the 41st International Conference on Machine Learning*. 2024. arXiv:2402.01622. https://arxiv.org/abs/2402.01622.

[23] H. S. Zheng et al. “NATURAL PLAN: Benchmarking LLMs on Natural Language Planning.” arXiv:2406.04520, 2024. https://arxiv.org/abs/2406.04520.

A Example prompt

The complete user message for business B001 (level 1 café) under California rules in the messages format. The system message was: “You are a scheduling assistant for a small business. You build weekly staff schedules.”

```
I run a cafe (California). We're open 06:00-21:00 every day. Please make next week's staff
    schedule.

RULES (all must be followed):
- Daily overtime: hours worked over 8 in a day are paid at 1.5x; hours over 12 in a day at 2x.
- Weekly overtime: hours worked over 40 in the week are paid at 1.5x.
- Seventh day: a worker who works all 7 days of the week is paid 1.5x for the first 8 hours of
     the 7th day.
- Meal breaks: anyone working more than 5 hours in a day must get an unpaid 30-minute meal
    break that starts before they have worked 5 hours; more than 10 hours requires a second
    one. No one has agreed to waive a meal break. A missed meal break costs 1 extra hour of
    pay.
- Workers aged 14-15: only outside school hours (school is Mon-Fri 8:00-15:00); at most 3
    hours on a school day and 18 hours in a school week; at most 8 hours on a non-school day;
    only between 7:00 and 19:00.
- Workers aged 16-17: only outside school hours; at most 4 hours on a school day, 8 hours on
    other days, 48 hours a week; only between 5:00 and 22:00, or until 00:30 on a night before
     a non-school day.
- My policy: do not schedule anything that would trigger overtime or any other premium pay.
- Each shift needs exactly the staff listed, in the roles listed. Each person can only fill a
    role they have, can only work when they are available for the whole shift, at most one
    shift per day, and no more than their max hours for the week.
- A 30-minute meal break is unpaid and does not count as hours worked.

STAFF
ID | Name | Age | Roles | Wage $/h | Max hours/week
E01 | Quinn | 32 | barista | 23 | 40
E02 | Uma | 53 | barista | 24 | 40
E03 | Bea | 46 | barista | 20 | 16
E04 | Dev | 57 | barista | 18 | 16
E05 | Cole | 36 | barista | 18 | 40

AVAILABILITY (messages from staff this week)
Quinn (E01): "can't do Mon; free Tue/Sun from 6am to 9pm; Wed 10am-9pm works; Thu/Fri/Sat 6am
    -2:30pm works"
Uma (E02): "Mon/Tue: 6am-2:30pm; Wed/Fri 6am-9pm works; free Thu/Sun from 10am to 9pm; can't
    do Sat!"
Bea (E03): "Hey! free Mon/Fri/Sun from 6am to 2:30pm; no Tue for me; Wed: 10am-9pm; Thu/Sat: 6
    am-9pm thx"
Dev (E04): "free Mon/Thu from 10am to 9pm; can't do Tue; free Wed/Fri/Sun from 6am to 9pm; Sat:
     6am-2:30pm"
Cole (E05): "Mon/Tue/Fri/Sun: 6am-9pm; Wed 6am-2:30pm works; Thu/Sat 10am-9pm works thx"

SHIFTS
ID | Day | Time | Needs
S01 | Mon | 06:00-14:30 | 1 barista
S02 | Mon | 12:30-21:00 | 1 barista
S03 | Tue | 06:00-14:30 | 1 barista
S04 | Tue | 12:30-21:00 | 1 barista
S05 | Wed | 06:00-14:30 | 1 barista
S06 | Wed | 12:30-21:00 | 1 barista
S07 | Thu | 06:00-14:30 | 1 barista
S08 | Thu | 12:30-21:00 | 1 barista
S09 | Fri | 06:00-14:30 | 1 barista
S10 | Fri | 12:30-21:00 | 1 barista
```

```
S11 | Sat | 06:00-14:30 | 1 barista
S12 | Sat | 12:30-21:00 | 1 barista
S13 | Sun | 06:00-14:30 | 1 barista
S14 | Sun | 12:30-21:00 | 1 barista

ANSWER FORMAT
Put the schedule between <schedule> and </schedule>, one line per person per shift:
EMPLOYEE_ID,SHIFT_ID,ROLE,MEAL
MEAL is the start time (HH:MM) of a 30-minute unpaid meal break, two separated by ';', or -
    for none.
Example: E03,S07,shift lead,12:00
If no schedule can follow every rule, put only IMPOSSIBLE between the tags and say why in one
    sentence.
```

B Reproducing the results

From the repository root:

- `python3 generate.py` recreates the 600 businesses.
- `python3 build_answer_key.py` recomputes the answer key.
- `python3 run.py` and `python3 solver_run.py` query the models (an OpenRouter key is required).
- `python3 grade.py results/main` grades the text-only replies, and `python3 grade.py results/solver` the code-tool replies.
- `python3 analysis.py` produces every table in this paper in `results/analysis/`.
- `python3 tests/test_checker.py` runs the grader tests.